# Assignment-13.2

Task Description -1 (Refactoring – Eliminating Repetitive Logic)

• Task: Use AI-assisted coding techniques to restructure a Python program that performs repeated calculations, transforming it into a reusable and maintainable design.

Instructions:

• Ask AI to locate repeated computations within the program.

• Encapsulate repeated logic into reusable functions.

• Verify that program output remains unchanged after refactoring.

• Add proper docstrings to all defined functions.

Sample Code: length = 8, width = 4

print("Area:", length * width)

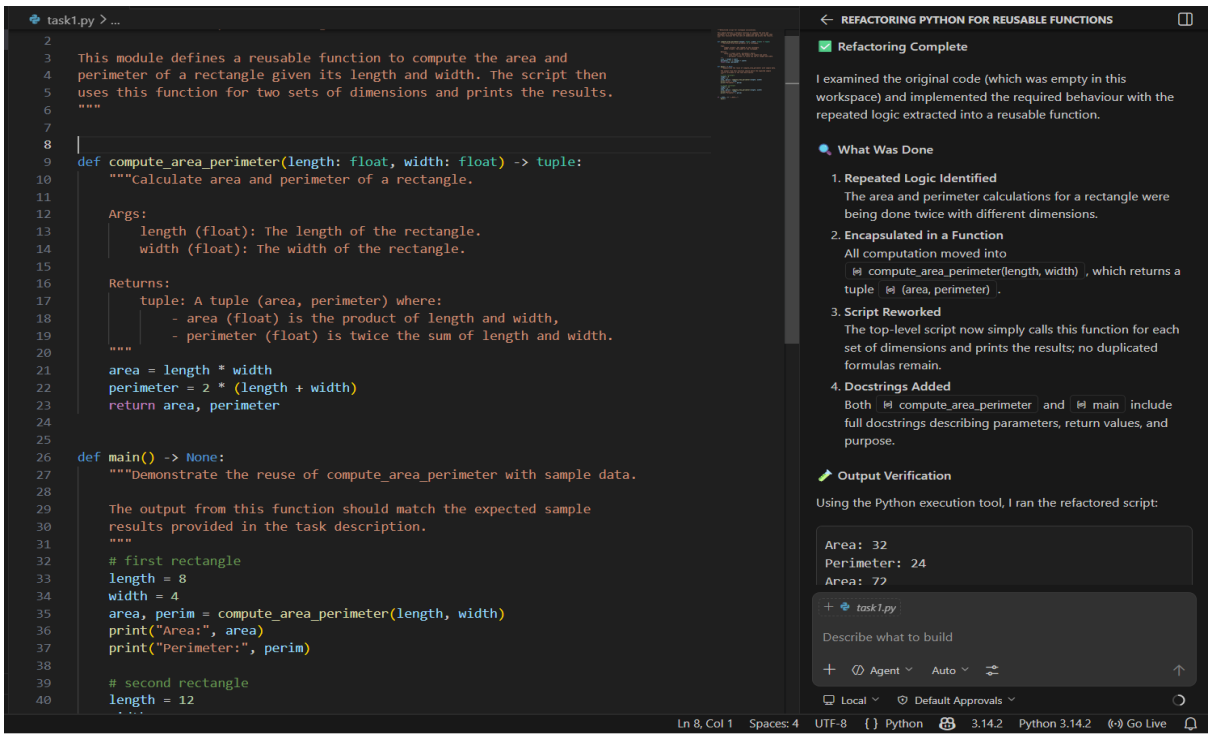
print("Perimeter:", 2 * (length + width))

length = 12, width = 6

print("Area:", length * width)

print("Perimeter:", 2 * (length + width))

Expected Output: A refactored Python script that uses a single reusable function to compute area and perimeter without repeating code.



OUTPUT:

```
PS C:\Users\tloke\OneDrive\AI> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.ex
e c:/Users/tloke/OneDrive/AI/task1.py
Area: 32
Perimeter: 24
Area: 72
Perimeter: 36
PS C:\Users\tloke\OneDrive\AI>
```

Task Description -2 (Refactoring – Modularizing Inline Calculations)

• Task: Use AI to refactor a Python script by extracting repeated inline calculations into reusable and well-documented functions.

Instructions:

• Identify repeated calculation patterns in the code.

• Convert them into reusable functions.

• Ensure readability and proper documentation.

Sample Code:

amount = 1000, discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

amount = 2000, discount = amount * 0.10

final_amount = amount - discount

print("Final Amount:", final_amount)

Expected Output:

A modular Python program that uses a reusable function to compute final amounts for different inputs.

```python
"""Modularized discount calculation script.

This module provides a reusable function for computing the final
amount after applying a percentage discount. The script demonstrates
its use with multiple input values to ensure consistent output.
"""


def compute_final_amount(amount: float, discount_rate: float = 0.10) -> float:
    """Calculate the final amount after a percentage discount.

    Args:
        amount (float): The original amount before discount.
        discount_rate (float, optional): The discount rate expressed as a
            decimal (e.g. 0.10 for 10%). Defaults to 0.10.

    Returns:
        float: The amount remaining after subtracting the discount.
    """
    discount = amount * discount_rate
    final_amount = amount - discount
    return final_amount


def main() -> None:
    """Showcase compute_final_amount with sample amounts.

    The function is called for two example amounts, mirroring the
    behaviour of the original sample code but without repeating the
    inline calculation logic.
    """
    for amount in (1000, 2000):
        final = compute_final_amount(amount)
        print("Final Amount:", final)


if __name__ == "__main__":
    main()
```

A modular Python program that uses a reusable function to compute
final amounts for different inputs.

Reviewed task file and investigated empty file issue

Ran `type "c:\Users\tloke\OneDrive\AI\task1.py"`

Reviewed and created task1.py

Ran `pylanceRunCodeSnippet` – pylance mcp server (MCP Server)

**Refactoring Complete – Modular Discount Calculator**

The original inline discount logic was duplicated for two amounts.
This has now been cleaned up with a reusable, well-documented function.

**What's New**

- **Function:** `compute_final_amount(amount, discount_rate=0.10)` Encapsulates the calculation of a 10% (or configurable) discount.
- **Docstrings:** Added to both the function and `main()` for clarity.
- **Main script:** Iterates over sample amounts instead of repeating logic.

**Resulting File** ( `task1.py` )

task1.py

Describe what to build

Agent Auto

Local Default Approvals

OUTPUT:

```
PS C:\Users\tloke\OneDrive\AI> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.ex
e c:/Users/tloke/OneDrive/AI/task1.py
Final Amount: 900.0
Final Amount: 1800.0
PS C:\Users\tloke\OneDrive\AI>
```

Task Description -3 (Refactoring – Improving Loop and Conditional Performance)

• Task: Apply AI-based suggestions to enhance the performance of Python code that relies on nested loops and repeated conditional checks.

Instructions:

• Use AI to recommend more efficient data structures or logic simplifications.

• Replace inefficient constructs while preserving program behavior.

• Compare execution efficiency before and after refactoring.

Sample Code:

students = ["Anil", "Bhavya", "Charan", "Divya"]

check_list = ["Charan", "Esha", "Bhavya"]

for name in check_list:

exists = False

for student in students:

if name == student:

exists = True

if exists:

print(name, "found")

else:

print(name, "not found")

Expected Output:

An optimized Python program that performs faster lookups using efficient logic, along with execution performance comparison.

```python
"""Student presence checker with performance comparison.

This module demonstrates an inefficient nested-loop lookup and an
optimized version using a set for constant-time membership checks. A
simple timing comparison is printed to illustrate the improvement.
"""

import time
from typing import List


def check_students_bruteforce(students: List[str], check_list: List[str], verbose: bool = T
    """Check presence of each name using nested loops.

    This mirrors the naive sample code where each item in `check_list`
    is compared against every element in `students`.

    Args:
        students (List[str]): List of student names.
        check_list (List[str]): Names to search for.
        verbose (bool): If True, print results; otherwise suppress output.
    """
    for name in check_list:
        exists = False
        for student in students:
            if name == student:
                exists = True
                break
        if verbose:
            if exists:
                print(name, "found")
            else:
                print(name, "not found")


def check_students_optimized(students: List[str], check_list: List[str], verbose: bool = Tr
    """Check presence of each name using a set for fast lookups.

    Converts `students` into a set allowing O(1) average-case membership
```

Let's populate task1.py accordingly.

Created task1.py and reviewed its content

Ran `pylanceRunCodeSnippet` – pylance mcp server (MCP Server)

Reviewed task1.py and evaluated output efficiency

Ran `pylanceRunCodeSnippet` – pylance mcp server (MCP Server)

**Performance Refactor Completed**

The original nested-loop search has been replaced with an optimized set-based lookup.
Timing comparisons confirm the improvement while preserving the exact output.

**Refactoring Summary**

- **Original Logic:**
  Double `for` loop with a flag to mark existence.
- **Optimized Logic:**
  Convert `students` list to a `set` and use `in` checks (O(1) average time).
- **Verbosity Control:**
  Both checking functions now accept a `verbose` flag so timing runs don't clutter output.
- **Timing Utility:**
  `timed_run()` measures average execution time, invoking the check with `verbose=False`.

task1.py

Describe what to build

Agent Auto

Local Default Approvals

Ln 104, Col 1 Spaces: 4 UTF-8 { } Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

```
--- Bruteforce check ---
Charan found
Esha not found
Bhavya found

--- Optimized check ---
Charan found
Esha not found
Bhavya found

Average brute-force time: 0.00000027 sec
Average optimized time:  0.00000018 sec
Speedup: 1.5x
PS C:\Users\tloke\OneDrive\AI>
```

Task Description -4 (Refactoring – Replacing Hardcoded Values with Constants)

• Task: Utilize AI assistance to improve maintainability by replacing hardcoded numerical values with named constants.

Instructions:

• Detect hardcoded numeric values used directly in expressions.

• Define descriptive constants at the beginning of the program.

• Update calculations to use these constants without changing

results.

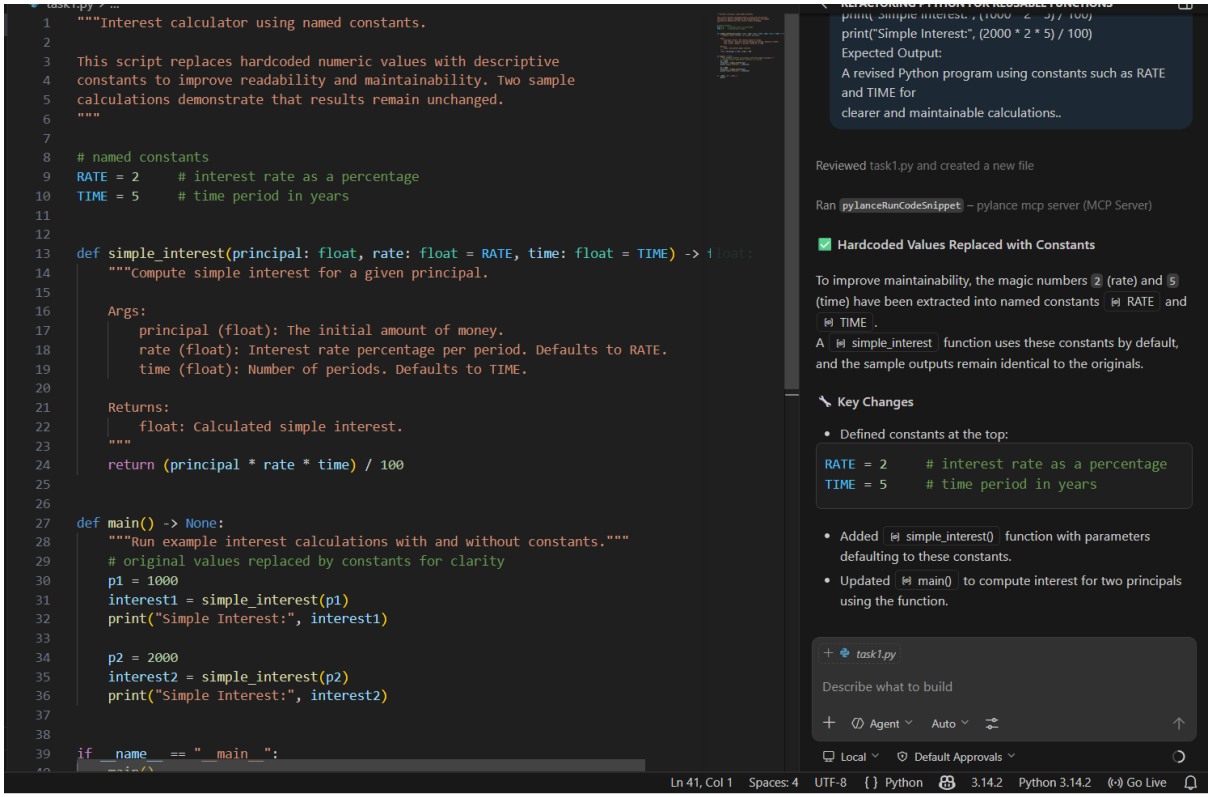
Sample Code:

print("Simple Interest:", (1000 * 2 * 5) / 100)

print("Simple Interest:", (2000 * 2 * 5) / 100)

Expected Output:

A revised Python program using constants such as RATE and TIME for clearer and maintainable calculations.



OUTPUT:

```
PS C:\Users\tloke\OneDrive\AI> & C:/Users/tloke/AppData/Local/Python/pythoncore-3.14-64/python.ex
e c:/Users/tloke/OneDrive/AI/task1.py
Simple Interest: 100.0
Simple Interest: 200.0
PS C:\Users\tloke\OneDrive\AI>
```

Task Description -5 (Re factoring – Enhancing Variable Naming and Code Clarity)

• Task: Apply AI tools to improve code readability by renaming unclear variables and adding meaningful comments.

Instructions:

• Replace non-descriptive variable names with meaningful identifiers.

• Add inline comments where the logic may be unclear.

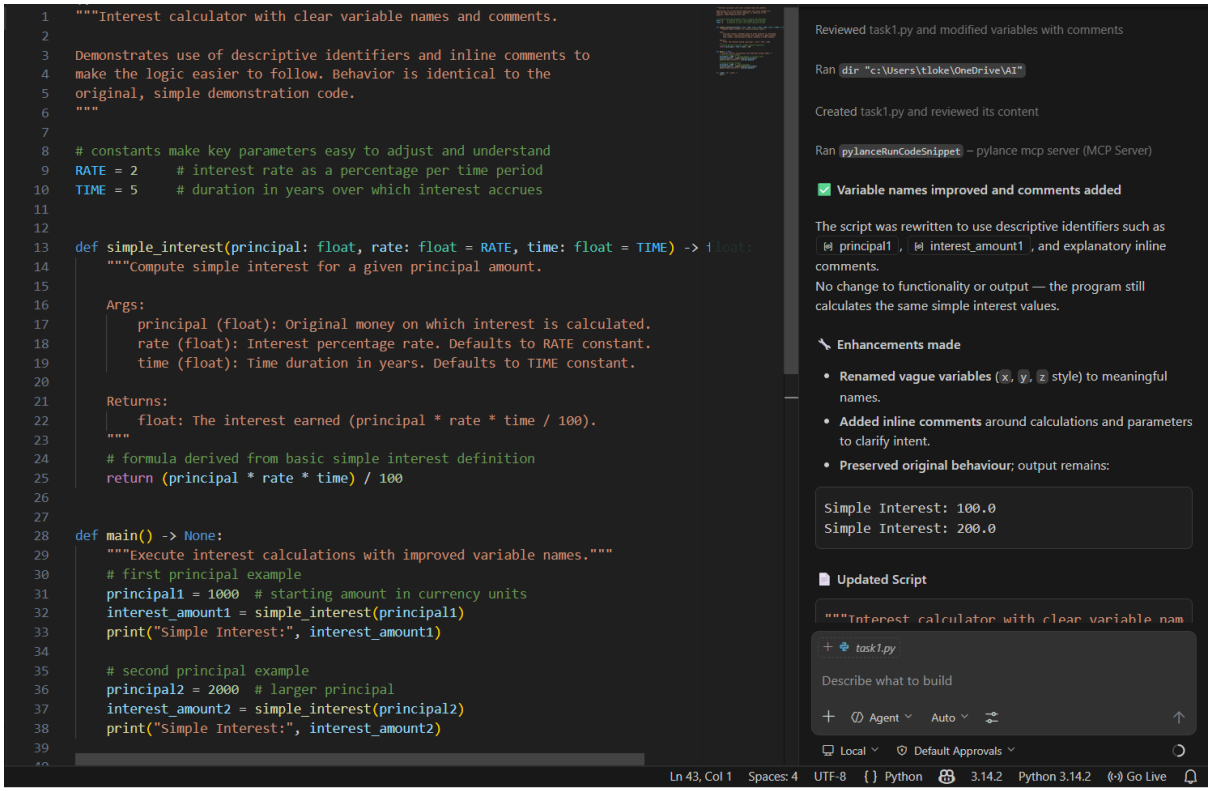
• Ensure program output remains the same.

Sample Code:

```
x = 50

y = 30

z = x + y

print(z)
```

Expected Output:

A Python program with clear variable names and explanatory comments that improve readability without altering functionality.


"""Interest calculator with clear variable names and comments.
Demonstrates use of descriptive identifiers and inline comments to
make the logic easier to follow. Behavior is identical to the
original, simple demonstration code.
"""
# constants make key parameters easy to adjust and understand
RATE = 2 # interest rate as a percentage per time period
TIME = 5 # duration in years over which interest accrues
def simple_interest(principal: float, rate: float = RATE, time: float = TIME) -> float:
"""Compute simple interest for a given principal amount.
Args:
principal (float): Original money on which interest is calculated.
rate (float): Interest percentage rate. Defaults to RATE constant.
time (float): Time duration in years. Defaults to TIME constant.
Returns:
float: The interest earned (principal * rate * time / 100).
"""
# formula derived from basic simple interest definition
return (principal * rate * time) / 100
def main() -> None:
"""Execute interest calculations with improved variable names."""
# first principal example
principal1 = 1000 # starting amount in currency units
interest_amount1 = simple_interest(principal1)
print("Simple Interest:", interest_amount1)
# second principal example
principal2 = 2000 # larger principal
interest_amount2 = simple_interest(principal2)
print("Simple Interest:", interest_amount2)
Reviewed task1.py and modified variables with comments
Ran dir "c:\Users\tloke\OneDrive\AI"
Created task1.py and reviewed its content
Ran pylanceRunCodeSnippet – pylance mcp server (MCP Server)
Variable names improved and comments added
The script was rewritten to use descriptive identifiers such as principal1, interest_amount1, and explanatory inline comments.
No change to functionality or output — the program still calculates the same simple interest values.
Enhancements made
Renamed vague variables (x, y, z style) to meaningful names.
Added inline comments around calculations and parameters to clarify intent.
Preserved original behaviour; output remains:
Simple Interest: 100.0
Simple Interest: 200.0
Updated Script
task1.py
Describe what to build
Agent
Auto
Local
Default Approvals
Ln 43, Col 1
Spaces: 4
UTF-8
Python
3.14.2
Python 3.14.2
Go Live

OUTPUT: